

Hora de tentar sozinhos, agora com Random Forest

Mesmo dataset mpg, mesma categoria economico — agora aplicando Random Forest.

Igual à Árvore de Decisão, não precisa normalizar. A novidade é o `n_estimators` — quantas árvores usar na floresta.

Escolham 2 ou mais features, do jeito que já fizeram nas práticas anteriores.



Importações necessárias

RandomForestClassifier, não mais DecisionTreeClassifier.

LEMBRETE DE SINTAXE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```



Categoria e features

Recriem economico, olhem a correlação, e escolham 2 ou mais features.

LEMBRETE DE SINTAXE

```
df = pd.read_csv("mpg_desafio_alunos.csv")
df["economico"] = (df["mpg"] >= 23).astype(int)

df.corr(numeric_only=True)["economico"]

X = df[["weight", "cylinders"]] # escolham as de vocês
y = df["economico"]
```



Tratar valores nulos

Necessário só se horsepower estiver entre as escolhidas.

LEMBRETE DE SINTAXE

```
df = df.dropna(subset=["horsepower"])
```



Separar treino e teste

stratify=y de novo, igual sempre.

LEMBRETE DE SINTAXE

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=5, stratify=y  
)
```



Treinar e avaliar

Comecem com `n_estimators=100`, um valor padrão razoável.

LEMBRETE DE SINTAXE

```
modelo = RandomForestClassifier(n_estimators=100, random_state=42)
modelo.fit(X_train, y_train)

pred = modelo.predict(X_test)
print(f"Acuracia: {accuracy_score(y_test, pred):.2%}")
```

Testar diferentes n_estimators

Vejam se a acurácia estabiliza, e a partir de quantas árvores.

LEMBRETE DE SINTAXE

```
valores_n = [1, 5, 10, 25, 50, 75, 100, 150, 200]
acuracias = []
for n in valores_n:
    m = RandomForestClassifier(n_estimators=n, random_state=42)
    m.fit(X_train, y_train)
    acuracias.append(accuracy_score(y_test, m.predict(X_test)))

plt.plot(valores_n, acuracias, marker="o")
plt.show()
```



Ver importância das features

Comparem com o que a Árvore de Decisão única mostrou — ficou mais distribuído?

LEMBRETE DE SINTAXE

```
importancias = pd.Series(  
    modelo.feature_importances_, index=X.columns  
)  
.sort_values()  
  
importancias.plot(kind="barh")  
plt.show()
```


Avaliação final

Rodem no mpg_desafio_final.csv, com a mesma coluna economico calculada.

LEMBRETE DE SINTAXE

```
df_final = pd.read_csv("mpg_desafio_final.csv")
df_final["economico"] = (df_final["mpg"] >= 23).astype(int)

X_final = df_final[["weight", "cylinders"]]
y_final = df_final["economico"]

pred_final = modelo.predict(X_final)
print(f"Acuracia final: {accuracy_score(y_final, pred_final):.2%}")
```



Refletir

Comparando os 4 algoritmos praticados (Regressão Logística, KNN, Árvore, Random Forest): a acurácia final melhorou de fato com Random Forest, ou ficou parecida com a árvore única?

Nem sempre 'mais complexo' significa 'muito melhor' — depende do problema.

Bônus opcional

Pra quem terminar cedo: testem também `max_features` ("sqrt", "log2", ou um número) e vejam se muda a acurácia ou a importância das features.

```
RandomForestClassifier(n_estimators=100, max_features='sqrt')
```



4 algoritmos praticados, mesmo dataset

Features escolhidas, floresta treinada, n_estimators testado, avaliada no conjunto nunca visto — comparando com todos os algoritmos anteriores.

A seguir: métricas completas (precisão, recall, F1) e validação cruzada.